

JAVA PROGRAMMING EXPERIMENT

Basic Information

Experiment No.	03
Date	15/09/2026
Title	Digital Payment System using Interface and Custom Exceptions

Objective / Aim

To design and implement a Digital Payment System in Java using an interface, encapsulation, and user-defined checked exceptions. The program simulates wallet creation, adding money, UPI payments, balance checking, and transaction validation.

System Requirements

Hardware Requirements

S.No.	Hardware Requirement
1	Standard PC or Laptop
2	Processor: Intel Core i3/i5 or equivalent AMD processor
3	RAM: Minimum 4 GB
4	Storage: Minimum 10 GB free space
5	Keyboard and Mouse
6	Monitor / Display

Software Requirements

S.No.	Software Requirement
1	Operating System: Windows / Linux / macOS
2	Java Development Kit (JDK): JDK 17 or JDK 21
3	Java Compiler: javac
4	Java Runtime: java
5	IDE: Eclipse / IntelliJ IDEA / NetBeans / VS Code
6	Command Prompt / Terminal

Algorithm / Logic Description

1. Define the PaymentService interface with pay() and checkBalance() methods.
2. Create three custom checked exception classes: InvalidUPIException, InvalidAmountException, and InsufficientBalanceException.
3. Create the Wallet class with private user details and balance to demonstrate encapsulation.
4. Initialize wallet details using a constructor.
5. In addMoney(), reject zero or negative amounts by throwing InvalidAmountException; otherwise add the amount to the balance.
6. Create UPIPayment that implements PaymentService and stores a Wallet object.
7. In pay(), validate the receiver UPI ID. It must contain '@'; otherwise throw InvalidUPIException.
8. Validate the payment amount. If it is not greater than zero, throw InvalidAmountException.
9. Compare the payment amount with the wallet balance. If it is greater, throw InsufficientBalanceException.
10. Deduct the payment amount and display the successful transaction.
11. In main(), create the wallet, add Rs.500, check the balance, and pay Rs.700 to rahul@upi.
12. Handle the custom exceptions using separate catch blocks and execute the finally block.
13. Display the final wallet details and final balance.

UML Class Diagram



Source Code

```
// Interface
interface PaymentService {

    // Method to make payment
    void pay(String receiverUPI, double amount)
        throws InvalidUPIException,
               InvalidAmountException,
               InsufficientBalanceException;

    // Method to check balance
    void checkBalance();
}

// Custom Exception 1
class InsufficientBalanceException extends Exception {

    // Constructor
    InsufficientBalanceException(String message) {
        super(message);
    }
}

// Custom Exception 2
class InvalidUPIException extends Exception {

    // Constructor
    InvalidUPIException(String message) {
        super(message);
    }
}

// Custom Exception 3
class InvalidAmountException extends Exception {

    // Constructor
    InvalidAmountException(String message) {
        super(message);
    }
}

// Wallet class
class Wallet {

    // Private data members for encapsulation
    private String userName;
    private String mobileNumber;
    private String upiId;
    private double balance;

    // Constructor to initialize wallet details
    Wallet(String userName, String mobileNumber,
           String upiId, double balance) {

        this.userName = userName;
        this.mobileNumber = mobileNumber;
        this.upiId = upiId;
        this.balance = balance;
    }

    // Method to add money
    void addMoney(double amount) throws InvalidAmountException {

        // Check whether amount is valid
        if (amount <= 0) {
            throw new InvalidAmountException(
                "Invalid amount. Amount must be greater than zero.");
        }

        // Add money to balance
        balance = balance + amount;

        System.out.println("Money added successfully: Rs." + amount);
    }

    // Method to get current balance
    double getBalance() {
        return balance;
    }

    // Method to get UPI ID
    String getUpiId() {
        return upiId;
    }
}
```

```

// Method to display wallet details
void displayWalletDetails() {

    System.out.println("\n===== WALLET DETAILS =====");
    System.out.println("User Name      : " + userName);
    System.out.println("Mobile Number: " + mobileNumber);
    System.out.println("UPI ID       : " + upiId);
    System.out.println("Balance      : Rs." + balance);
}

}

// UPIPayment class implements interface
class UPIPayment implements PaymentService {

    // Wallet object
    private Wallet wallet;

    // Constructor
    UPIPayment(Wallet wallet) {
        this.wallet = wallet;
    }

    // Implementing pay() method
    public void pay(String receiverUPI, double amount)
        throws InvalidUPIException,
               InvalidAmountException,
               InsufficientBalanceException {

        // Validate UPI ID
        // UPI ID must contain '@'
        if (receiverUPI == null ||
            !receiverUPI.contains("@")) {

            throw new InvalidUPIException(
                "Invalid UPI ID. Payment cancelled.");
        }

        // Validate payment amount
        if (amount <= 0) {

            throw new InvalidAmountException(
                "Invalid payment amount. Amount must be greater than zero.");
        }

        // Check available balance
        if (amount > wallet.getBalance()) {

            throw new InsufficientBalanceException(
                "Insufficient balance. Payment cancelled.");
        }

        // Deduct amount from wallet
        // We use addMoney() with negative amount is not possible,
        // so payment deduction is handled through this method.
        deductAmount(amount);

        // Display successful transaction
        System.out.println("\nPayment Successful!");
        System.out.println("Receiver UPI : " + receiverUPI);
        System.out.println("Amount Paid  : Rs." + amount);
    }

    // Private method to deduct money
    private void deductAmount(double amount) {

        try {
            // Use reflection to update private balance
            java.lang.reflect.Field field =
                wallet.class.getDeclaredField("balance");

            field.setAccessible(true);

            double currentBalance =
                (double) field.get(wallet);

            field.set(wallet, currentBalance - amount);

        } catch (Exception e) {
            System.out.println("Unable to process payment.");
        }
    }

    // Implementing checkBalance() method
    public void checkBalance() {

        System.out.println(
            "Available Balance: Rs." + wallet.getBalance());
    }

}

```

```

// Main class
public class DigitalPaymentDemo {

    public static void main(String[] args) {

        // Create Wallet object
        Wallet wallet = new Wallet(
            "Ashiya",
            "9876543210",
            "ashiya@upi",
            1000.0
        );

        // Create UPIPayment object
        UPIPayment payment = new UPIPayment(wallet);

        // Display initial wallet details
        wallet.displayWalletDetails();

        try {

            // Add money to wallet
            wallet.addMoney(500);

            // Check balance
            payment.checkBalance();

            // Make UPI payment
            payment.pay("rahul@upi", 700);

        } catch (InvalidUPIException e) {

            // Handle invalid UPI exception
            System.out.println("Transaction Error: "
                + e.getMessage());

        } catch (InvalidAmountException e) {

            // Handle invalid amount exception
            System.out.println("Transaction Error: "
                + e.getMessage());

        } catch (InsufficientBalanceException e) {

            // Handle insufficient balance exception
            System.out.println("Transaction Error: "
                + e.getMessage());

        } finally {

            // This block always executes
            System.out.println("\nTransaction process completed.");

        }

        // Display final wallet details
        wallet.displayWalletDetails();

        // Display final balance
        payment.checkBalance();

    }
}

```

Completion and Execution Commands

Save the source code as **DigitalPaymentDemo.java**. The filename must match the public class name.

1. Open Command Prompt / Terminal.
2. Navigate to the folder containing DigitalPaymentDemo.java.
3. Compile:
javac DigitalPaymentDemo.java
4. Run:
java DigitalPaymentDemo
5. Check the console for the transaction output.

Output Screenshot / Console Output

Expected output for the supplied program. Take the actual screenshot from your IDE or terminal and place it in the experiment record if required.

```
===== WALLET DETAILS =====
User Name      : Ashiya
Mobile Number: 9876543210
UPI ID         : ashia@upi
Balance        : Rs.1000.0
Money added successfully: Rs.500.0
Available Balance: Rs.1500.0
```

```
Payment Successful!
Receiver UPI : rahul@upi
Amount Paid  : Rs.700.0
```

```
Transaction process completed.
```

```
===== WALLET DETAILS =====
User Name      : Ashiya
Mobile Number: 9876543210
UPI ID         : ashia@upi
Balance        : Rs.800.0
Available Balance: Rs.800.0
```

Result / Completion

The Digital Payment System was successfully implemented in Java. The program demonstrates interface implementation, encapsulation, constructors, custom checked exceptions, exception handling, wallet operations, UPI validation, and balance management.